Aula 0 — Como se programa um microcontrolador

Um chip nu, um kit e o mesmo LED

Microcontroladores — DENE/UFMT

OBJETIVOS

- Reconhecer o que distingue um microcontrolador de um computador de mesa, e identificar no chip nu os elementos mínimos para que ele execute código.
- Escrever a mesma lógica em Python e em C, e apontar o que desaparece na travessia: a biblioteca, o sistema operacional e o retorno de `main`.
- Explicar por que a saída de um programa embarcado é um pino, e por que escrever num pino é escrever numa posição de memória.
- Descrever a cadeia que leva do arquivo `.c` ao conteúdo da memória de programa, e distinguir os dois caminhos de gravação disponíveis.
- Situar o próprio trabalho no semestre: reconhecer o termostato como artefato final e o calendário de entregas e avaliações.

1 O curso

Meia hora desta aula é regra e calendário. Vale ler antes, porque quase toda reclamação de fim de semestre nasce de algo que estava escrito nesta seção.

1.1 Duas disciplinas, um objeto

Teoria e laboratório são duas disciplinas, com 32 horas e nota cada uma. São também a mesma coisa: o laboratório existe para construir aquilo que a teoria descreve, e a teoria existe porque o laboratório iria falhar sem ela. Você pode ser aprovado em uma e reprovado na outra, mas não vai passar em nenhuma tratando-as como assuntos separados.

Uma mudança em relação aos semestres anteriores: **o laboratório implementa a teoria da semana anterior**, não a do mesmo dia. Os dois primeiros roteiros, R0 e R1, são introdutórios e não valem nota; a partir do R2 o laboratório anda um encontro atrás da teoria. O motivo é dar uma semana de decantação entre ouvir e fazer — e permitir que o miniteste da semana seguinte cobre o que você **fez**, e não só o que ouviu.

1.2 O que vamos construir

O curso não é uma coleção de experimentos. É **um** artefato, construído em camadas: um termostato digital. Cada roteiro acrescenta uma camada ao mesmo código, e o código de referência é publicado depois de cada sessão — de modo que quem tropeçou numa semana não fica travado na seguinte.

Camada	O que passa a existir
R3	Um pino que liga e desliga em tempo controlado
R4	Uma tela que mostra números
R5	Uma temperatura lida do mundo real
R6	Um relógio que não depende do laço, e um contador de eventos
R7	Um programa que atende eventos na hora certa, sem esperar
R8	Uma saída proporcional, não apenas ligada ou desligada
R9	Uma decisão automática, e um vigia em silício que não depende dela
R10	Um registro do que aconteceu, enviado para fora
R11	Uma memória que sobrevive a desligar a placa
R13	Um controle que não fica oscilando em torno do alvo

TP	O termostato inteiro, com aquecedor e cooler por PWM, apresentado e defendido

Ao final, o objeto na sua bancada é o mesmo que existe dentro de uma estufa, de uma incubadora ou de um ar-condicionado inverter. Não é um modelo reduzido dele: é ele.

1.3 Calendário

#	Teoria	Laboratório	Miniteste
0	Como se programa um microcontrolador	R0 — C, do PC ao micro; binário e hexa	—
1	Memória e folha de dados	R1 — o kit XM118: componentes, medidas, bits	M1
2	PC e ciclo de busca-execução; o pino como endereço	R2 — classe de memória em Python	M2
3	Barramento paralelo e o controlador HD44780	R3 — LED, atraso e frequência (osciloscópio)	M3
4	Aquisição analógica: ADC e LM35	R4 — display HD44780 em quatro bits	M4
5	Temporizadores	R5 — entrada digital, ADC e LM35 no display	M5
6	**Avaliação integradora I**	R6 — temporizadores: medir e contar	—
7	Interrupções e ruído de contato	**Avaliação prática I — o painel**	M7
8	PWM e estágio de potência	R7 — interrupções, botões e repique	M8
9	Controle liga-desliga, histerese e comparação analógica	R8 — PWM na ventoinha e no buzzer	M9
10	Comunicação serial	R9 — liga-desliga, histerese e comparador	M10
11	Barramento serial síncrono: I²C e memória não volátil	R10 — telemetria por UART	M11
12	**Avaliação integradora II**	R11 — setpoint em memória não volátil	—
13	Controle embarcado na prática: do liga-desliga ao PI	**Avaliação prática II**	—
14	**Seminários comparativos**	R13 — sintonia contra planta simulada	—
15	**Seminários comparativos** e encerramento	**TP — o termostato**	—

Cada roteiro acompanha a aula de mesmo número; os encontros 6 e 12 são provas e não têm roteiro próprio. O encontro 15 é a apresentação do TP, o termostato que junta todos os roteiros. Não há semana de folga: se o semestre atrasar, o R13 é o que cede.

1.4 Como se avalia

Os minitestes ocupam os dez minutos finais da aula de teoria e cobram o que foi visto naquele dia e nos anteriores. Nunca cobram o que ainda não foi coberto: se uma questão parece pedir conteúdo futuro, é erro meu, e vale reclamar na hora.

Teoria **Laboratório**

Instrumento	Instrumento	Peso	Peso
Minitestes (10, contam os 8 melhores)	Roteiros R2–R11 e R13 (11, sem descarte)	20%	40%
Avaliação integradora I	Avaliação prática I — o painel	25%	15%
Avaliação integradora II	TP — o termostato	30%	15%
Seminário comparativo	Avaliação prática II	25%	30%
	R0 e R1		sem nota

As avaliações práticas e o TP são novidade. Metade do curso acontece na bancada, e até agora só a metade teórica era avaliada diretamente. A avaliação prática I, na semana 7, é a primeira metade do termostato — medir, contar e mostrar —, feita em grupo na sessão e apresentada ao professor no fim. A avaliação prática II cai uma semana depois da integradora II, com tarefa fechada e tempo contado: ler um valor, mostrar, acionar uma saída. O TP é o termostato inteiro, apresentado no encontro 15 e defendido por cada integrante.

No seminário comparativo, a nota do grupo é ajustada por arguição individual. Isso não é desconfiança: é a única forma honesta de diferenciar quem estudou a plataforma de quem estudou os slides do colega na véspera.

1.5 Regras de convivência

- O material de cada encontro é publicado na página do curso **antes** da aula. Vir com ele lido muda a aula de ditado para discussão.
- O código de referência de cada roteiro é publicado **depois** da sessão. Use-o para conferir, não para começar.
- Roteiro entregue sem previsão preenchida não é corrigido. A previsão vale pelo raciocínio, não pelo acerto — errar uma previsão bem argumentada não custa nota.
- Bancada é trabalho em dupla. Relatório é individual.

ATENÇÃO

Segurança de bancada, antes de qualquer coisa: a **garra de terra do osciloscópio vai num ponto de GND, e em nenhum outro lugar.** Presa num ponto de 12 V — LAMP, HEATER, COOLER — ela curto-circuita esses 12 V contra o terra da rede.

Cumprida essa regra, medir é seguro: com a garra no GND, a ponteira pode ir a qualquer ponto do circuito. Este curso é feito de medições, e a regra existe para que vocês possam medir à vontade, não para que evitem o instrumento.

O tratamento completo, com a placa na mão e a caixa de perigo que ele merece, está no R1. Aqui fica o aviso mínimo, porque alguns de vocês vão encostar num osciloscópio antes disso.

2 O que é um microcontrolador

2.1 A demonstração

EXPERIMENTO

Na mesa há dois objetos rodando **o mesmo programa**:

1. Um PIC18F4550 espetado numa protoboard, com fonte, um capacitor, um resistor e um LED. Nada mais.
2. O kit XM118, com display, teclado, relés, conversor USB, sensor de temperatura e quarenta outras coisas soldadas em volta de um PIC18F4550 idêntico.

O LED pisca igual nos dois. Vale olhar para os dois ao mesmo tempo por um minuto antes de continuar lendo.

A pergunta que a demonstração responde é "o que é um microcontrolador", e ela responde por subtração: é aquilo que sobra quando se tira tudo que está em volta. O kit é conveniente e é ele que vocês vão usar o semestre inteiro — mas ele esconde justamente a resposta, porque nele o microcontrolador é o componente que menos chama atenção.

2.2 O que o chip nu precisa

Elemento	Por que é obrigatório	O que acontece sem ele
V_{DD} e V_{SS} (dois pares)	Alimentação do núcleo e das portas	Não liga, ou liga de forma instável — os dois pares precisam estar conectados
Capacitor de desacoplamento (100 nF)	Fornece corrente nas transições rápidas que a fonte não acompanha	Funciona na bancada e falha aleatoriamente depois; o defeito mais difícil de achar da lista
MCLR em nível alto	MCLR baixo mantém o processador em reset	Fica travado em reset para sempre, sem nenhum sintoma visível
Fonte de sinal de relógio	Sem relógio não há ciclo de instrução	Nada executa. O 4550 tem oscilador interno, então este item é opcional

		— e é por isso que a protoboard não tem cristal
Resistor em série com o LED	Limita a corrente do pino	O pino entrega corrente até se danificar

CONCEITO

Note o que **não** está na lista: memória, barramento, conversor A/D, temporizador, porta serial. Não estão porque já estão dentro do encapsulamento. Essa é a definição operacional de microcontrolador — o computador inteiro num único componente, com periféricos junto.

O que está na lista é apenas o que o silício não consegue fabricar dentro de si: energia, uma referência de tempo, e a garantia de que ninguém o está segurando em reset.

2.3 O que o kit acrescenta

Dentro do chip	Fora, na placa
Memória de programa, RAM, EEPROM interna	EEPROM I²C externa
Conversor A/D, comparadores	Sensor LM35, potenciômetros
Temporizadores, PWM	Ventilador, lâmpada, aquecedor
Módulo USB (transceptor incluído)	Conector USB e proteção
Portas digitais	LEDs com resistores, chaves, teclado matricial, display, relés

CONCEITO

Nada no seu programa diz que RD0 é um LED. Essa informação não está no `xc.h`, não está na folha de dados do PIC18F4550, não está no compilador. Ela está numa chave de duas posições no painel da placa.

Toda camada que o kit acrescenta é uma camada que nenhuma ferramenta conhece — e é onde vão morar quase todos os defeitos deste semestre.

2.4 O computador que você já conhece

Vocês já escreveram C. O mesmo compilador, apontado para outro destino, produz algo com três diferenças que aparecem já no primeiro programa.

No computador	Aqui	Consequência no código
Existe sistema operacional	Não existe	Não há para onde retornar. `main` não termina: termina num laço infinito, ou o processador continua buscando instruções no que vier depois
Existe `printf`	Não existe	A saída é um pino. Escrever num pino é escrever numa posição de memória
O programa é carregado na RAM	O programa mora na Flash	Variáveis inicializadas precisam ser copiadas da Flash para a RAM antes de `main` — por um código que ninguém escreveu

NOTA

A tabela sugere uma fronteira nítida que hoje não existe. Um Raspberry Pi Pico tem mais RAM que os primeiros PCs; um ESP32 roda pilha TCP/IP e sistema de arquivos; o processador do seu celular é um computador que cabe numa unha. Contar transistores, megahertz ou quilobytes não separa mais as duas coisas.

A diferença que ainda separa é a **unidade de gerenciamento de memória**. Ela traduz endereços — o endereço que o programa usa não é o endereço físico — e é isso que permite processos isolados, memória virtual e um sistema operacional que sobrevive a um programa mal escrito.

Sem ela, um ponteiro errado corrompe qualquer coisa, inclusive o próprio sistema. O PIC18F4550 não tem nada disso: o endereço que você escreve é o endereço que existe, e não há ninguém entre os dois. É por isso que o simulador do encontro 2 consegue ser cinco linhas — e é a mesma razão pela qual um Cortex-A roda Linux e um Cortex-M não.

3 As camadas, e por que descemos todas

Boa parte de vocês já piscou um LED. Em Arduino, em ESP32, ou num STM32 pelo configurador gráfico. E a pergunta legítima, na primeira aula, é por que este curso vai fazer de um jeito mais trabalhoso.

A resposta não é que aquilo é para amadores. Não é: Arduino roda dentro de produto que se vende. A resposta é que existem camadas entre o seu `if` e o elétron, e um curso de microcontroladores é aquele em que se desce por todas.

Camada	O que ela oferece	Quem a escreveu
Seu programa	A lógica que você quer	Você
Framework ou HAL	**Comportamento**: acender um pino, ler um valor	Alguém que nunca viu a sua placa
Mapa de registradores	**Nomes**: `LATD` significa 0xF8C	O fabricante do chip
Endereço cru	Nada. Um número e um ponteiro	Você, se nada existisse
Registrador	Um endereço com significado elétrico	O projetista do silício
Pino	Um transistor ligado a um fio	A física

A camada do "endereço cru" merece ser vista uma vez. Sem nenhum arquivo de apoio, escrever num pino do PIC18F4550 é isto: CONCEITO

```
#define LATD  (*(volatile unsigned char *) 0x0F8C)

LATD = 0x01;
```

E num Arduino, exatamente a mesma forma:

```
#define PORTB (*(volatile unsigned char *) 0x25)
```

Isso não é AVR contra PIC. É a mesma linha nas duas plataformas. O que faz `LATDbits.LATD0 = 1` parecer amigável não é o chip: é o `xc.h` conter alguns milhares de linhas assim, escritas por outra pessoa.

Não é para você entender essa linha hoje, e vale dizer por quê. Ela empilha seis ideias, e nenhuma delas existe em Python: ATENÇÃO

um **endereço** como número; a **conversão** desse número para ponteiro; a **desreferência** do ponteiro; a palavra `volatile`, que proíbe o compilador de otimizar o acesso; a base **hexadecimal**; e a ideia de que um endereço pode **ser** um periférico, em vez de guardar um valor.

O curso desce essas seis, uma por vez. No encontro 1 esta linha fica legível, e no encontro 2 você escreve o outro lado dela — o pedaço de memória que responde quando alguém escreve em 0x0F8C.

É a mesma dívida do período do pisca: declarada agora, paga com data marcada.

`xc.h` não é uma HAL. Ele é a camada de baixo dessa tabela: dá nomes, não comportamento. CONCEITO

A diferença é observável. `digitalWrite(13, HIGH)` funciona em qualquer pino de qualquer placa — é comportamento, e é portátil. `LATDbits.LATD0 = 1` só

significa alguma coisa neste chip, neste pino. O código com HAL viaja; o nosso não.

E é exatamente por isso que um custa e o outro não.

3.1 O mesmo pisca, em três camadas

Como se escreve	Custo	O que está escondido
`digitalWrite(13, HIGH)`	dezenas de ciclos	Tabela de pinos, desligamento de PWM, interrupções, leitura-modificação-escrita
`IO_RD0_SetHigh()`	**um ciclo**	Nada: é uma macro que vira a linha de baixo
`LATDbits.LATD0 = 1`	**um ciclo**	Nada

CONCEITO

Abstração em tempo de compilação é grátis; em tempo de execução, se paga em ciclos.

A macro do configurador da Microchip desaparece na compilação: sobra a mesma instrução única. A função do Arduino existe em tempo de execução — ela consulta uma tabela para descobrir qual porta corresponde ao pino 13, verifica se há um temporizador de PWM ativo naquele pino e o desliga, protege a operação contra interrupção, e só então escreve.

Nada disso é desperdício: é o preço de a mesma linha funcionar em cem placas diferentes. Mas é preço, e saber quando ele importa é conteúdo de engenharia.

NOTA

O número exato depende da versão do núcleo e da plataforma, e é **medível**: um pino alternando num laço, o osciloscópio, e a conta. Se a curiosidade bater antes do encontro 2, o instrumento está no laboratório.

3.2 Três razões para descer

CONCEITO

Primeira: alguém escreve a abstração. `digitalWrite` é código C que alguém escreveu, e que alguém mantém. Um curso de engenharia forma quem escreve, não apenas quem consome — e para escrever a camada é preciso conhecer a de baixo.

Segunda: a abstração vaza exatamente quando importa. Ela esconde detalhes até o dia em que um deles decide o resultado, e nesse dia não há como consertar por cima. Alguns exemplos que este semestre vai encontrar de frente: o tempo de aquisição do conversor decide se um divisor resistivo pode ser lido (encontro 4); uma função de envio serial bloquear ou enfileirar decide se o controle funciona (encontro 10); um contador de milissegundos de dezesseis bits dá a volta e o programa mede o tempo errado (encontro 7).

Terceira: aqui os recursos não sobram. São 32 kB de programa e 2 kB de dados. Uma abstração que reserva um buffer por canal é uma decisão de projeto, não um detalhe de implementação.

ATENÇÃO

O que este curso não está dizendo. Não está dizendo para não usar biblioteca.

A posição, que vale para o semestre inteiro, é outra: **derive uma vez, e depois use para sempre.** No encontro 10 vocês vão calcular à mão o divisor de uma taxa de comunicação — para que, quando o configurador mostrar uma caixa escrita “9600”, vocês saibam que ele programou 9592,3, por que programou isso, e se o erro cabe.

Quem nunca fez a conta não consegue julgar a caixa. E quem não consegue julgar a caixa não escolheu a biblioteca: aceitou.

NOTA

Uma consequência agradável, para o fim do semestre: quem escreve `LATDbits.LATD0 = 1` sabendo o que acontece está em condição de **escrever** um `digitalWrite`.

Essa é a diferença entre usar uma plataforma e ter uma.

4 O mesmo programa em três formas

O escopo de hoje é um LED que pisca. Nada além disso. O ponto não é a dificuldade do programa: é o que muda entre as três versões.

4.1 Em Python

```python
import time

while True:
    print("aceso")
    time.sleep(0.5)
    print("apagado")
    time.sleep(0.5)
```

Duas coisas aqui são serviços que alguém prestou a você. `print` supõe um terminal, um sistema operacional e uma biblioteca. `time.sleep` supõe um relógio mantido por esse mesmo sistema, e que existe outra coisa para o processador fazer enquanto você espera.

4.2 Em C, no computador

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    while (1) {
        printf("aceso\n");   usleep(500000);
        printf("apagado\n"); usleep(500000);
    }
    return 0;
}
```

Mudou a sintaxe. Não mudou nenhuma das duas suposições.

4.3 Em C, no microcontrolador

```c
#include <xc.h>

void main(void)
{
    unsigned int i, j;

    LATD  = 0x00;          // primeiro decide o valor
    TRISD = 0x00;          // depois deixa o pino sair

    while (1) {
        LATDbits.LATD0 = 1;
        for (j = 0; j < 10; j++) for (i = 0; i < 30000; i++);
        LATDbits.LATD0 = 0;
        for (j = 0; j < 10; j++) for (i = 0; i < 30000; i++);
    }
}
```

Era	Virou	Porque
`printf`	`LATDbits.LATD0`	A saída é um pino, e o pino é um nome que o compilador traduz para um endereço de memória
`usleep`	um laço contado	Não há sistema operacional para acordar você; a única forma de esperar é gastar instruções

`return 0`	`while (1)` Não existe destino para o retorno

`LATD` não é uma variável do seu programa. É um nome para uma posição fixa da memória, e aquela posição está fisicamente ligada aos pinos. A atribuição `LATDbits.LATD0 = 1` é uma escrita em memória como qualquer outra — só que essa escrita move elétrons num fio que sai do encapsulamento.

CONCEITO

Este é o fio condutor do primeiro terço do curso. No encontro 1 vamos descobrir **qual** endereço. No encontro 2, vamos escrever nele à mão.

`LAT` antes de `TRIS`. Por enquanto, decore: primeiro você escreve o valor que o pino deve ter, depois libera o pino para sair. A ordem inversa faz o pino assumir por alguns instantes o valor que estava lá antes — o que é irrelevante com um LED e é sério com um relé de 12 V ligado a um aquecedor.

ATENÇÃO

No encontro 2 isso deixa de ser regra decorada e passa a ser consequência.

No XM118, os LEDs **acendem** quando a chave do painel é fechada, com o processador ainda em reset e sem que uma linha de C tenha executado — pinos em reset ficam em alta impedância. Portanto o evento observável do seu primeiro programa é **apagar**, não acender. Se você gravar esperando ver um LED acender, vai concluir que não funcionou.

NOTA

4.4 Um atraso cuja duração você não sabe

Repare no que acabamos de fazer: escrevemos um programa cujo período ninguém nesta sala sabe calcular. Isso é deliberado.

Para saber o período faltam dois números: quantos ciclos de máquina custa uma iteração do laço, e quanto tempo dura um ciclo. Nenhum dos dois está no código, e nenhum dos dois vamos obter hoje.

Repare também que o laço é **duplo**. Um contador de 16 bits não chega perto de um pisca visível nesta plataforma, e você não tem como saber disso antes de tentar. Na bancada você vai ajustar os dois valores por tentativa até enxergar o LED comutar — que é a única técnica disponível para quem não sabe contar ciclos. Anote o par que funcionou.

A dívida. No encontro 2, com a listagem gerada pelo compilador na mão e o valor de τ_{cy}, você calcula o período que hoje só chutou. No R3, com o osciloscópio, você mede e descobre de quanto errou.

NOTA

Anote o palpite de hoje. Sem ele, o encontro 2 vira exercício de aritmética; com ele, vira conferência.

Existe uma macro `__delay_ms` no XC8 que parece resolver isso. Não vamos usá-la hoje. Ela exige uma constante conhecida em tempo de compilação e depende de um símbolo que você precisa definir com o valor certo — e explicar por quê consome quinze minutos de um assunto que não é o de hoje. É a primeira seção do encontro 2.

Um laço vazio pode ser removido pelo compilador: ele não produz efeito observável, e o compilador tem licença para descartá-lo. Nas nossas condições ele sobrevive, mas o sintoma de quando não sobrevive é bonito e vale reconhecer — o LED não pisca, e sim fica com meio brilho. No encontro 2, ao ler a listagem, você vai poder verificar se o laço ainda está lá.

NOTA

5 Como o código entra no chip

5.1 Do arquivo ao conteúdo da Flash

```
main.c → [pré-processador] → [compilador] → [linker + biblioteca] → main.hex
```

O `.hex` não é binário: é texto. Cada linha diz quantos bytes vêm, em qual endereço eles devem ser escritos, de que tipo é o registro, os bytes em si, e um byte de verificação. Abrir esse arquivo num editor de texto é a primeira coisa que faremos no R0 — junto

com as representações binária e hexadecimal, que existem porque são os dígitos dos endereços.

5.2 Dois caminhos

Você vai usar o caminho fácil o semestre inteiro. É justo saber que existe outro, e o que o fácil esconde.

	Bootloader USB	**Gravador (MPLAB Snap)**
Como	Segurar SW9 e ligar; um aplicativo no PC envia o `.hex` pela USB	Cinco fios em MCLR/V_{PP}, PGC, PGD, V_{DD}, V_{SS}
Hardware extra	Nenhum	O gravador
Exige	Que o bootloader já esteja gravado no chip	Nada; funciona em chip virgem
Grava a configuração	Não	Sim
Ocupa memória	Sim, o começo da Flash	Não

CONCEITO

O bootloader não é mágica. É um programa comum, gravado na Flash, que roda antes do seu e decide entre entregar o controle a você ou ficar escutando a USB. Quem escreve na Flash do PIC é o próprio PIC.

Isso também explica por que o chip nu da protoboard exigiu o gravador: nele não há ninguém escutando.

5.3 O código que veio antes do seu

Vale parar aqui, porque o que acabamos de descrever não é uma peculiaridade desta placa. É a situação normal de todo sistema computacional que você vai encontrar na vida.

CONCEITO

Seu programa quase nunca é o primeiro a rodar. Há código que executou antes, que decidiu entregar o controle a você, e que continua sendo dono de alguns recursos. Você não programa numa máquina vazia: programa no espaço que sobrou.

Onde	**Quem rodou antes e o que ele guardou para si**
XM118	Bootloader HID: o começo da Flash, os bits de configuração e os endereços de desvio
Placa STM32	Bootloader DFU em ROM: uma região de memória e a decisão de qual imagem carregar
Computador	Firmware UEFI: a inicialização do hardware, tabelas de memória e a ordem de carga
Celular	Uma cadeia de bootloaders, cada um verificando a assinatura do seguinte

A consequência prática é sempre a mesma: **existem coisas que o seu programa não pode mudar, porque não são dele.** Descobrir quais são, e por quê, é parte de conhecer qualquer plataforma.

Neste curso essa restrição vai aparecer pelo menos quatro vezes, sempre com a mesma assinatura — uma linha de código que parece correta, não dá erro nenhum, e não tem efeito.

NESTE KIT

O bootloader do XM118 grava a configuração de fábrica e não a reescreve. Logo:

o `#pragma config` da sua aplicação é **silenciosamente ignorado**;

a frequência do núcleo é 16 MHz e você não escolhe (encontro 2);

`PBADEN` deixa RB0–RB4 analógicos após o reset e você desfaz isso em tempo de execução, não na configuração (encontro 4);

o watchdog pode estar fora do seu alcance (encontro 5);

os endereços de desvio de interrupção pertencem a ele e precisam ser reencaminhados (encontro 7).

Nada disso vale para o chip nu da protoboard, e nada disso é conteúdo sobre microcontroladores: é conteúdo sobre **esta** placa. O que é conteúdo é o princípio acima.

NESTE KIT

É **SW9** que entra em modo bootloader, não SW1. O manual da Exsto diz SW1 e está errado; a serigrafia da placa tem precedência sobre o manual.

ATENÇÃO

Como o bootloader não grava configuração, os `#pragma config` que você escrever no seu código são **silenciosamente ignorados**. A configuração já está gravada, e foi ele quem escolheu: 48 MHz para a USB e **16 MHz para o núcleo**.

Guarde o número. Você não escolhe o relógio deste kit, e todo cálculo de tempo do semestre depende dele.

6 O fio condutor

Os quatro primeiros encontros abrem, um por vez, as camadas de um simulador do PIC18 escrito em Python:

usa o simulador (0) → abre a memória dele (1) → abre a execução (2) → confronta com um periférico real (3)

Cada camada aberta é imediatamente confrontada com a bancada. O simulador é determinístico e mostra o que o osciloscópio mostra rápido demais; a bancada mostra o que o simulador mente. Saber **onde** o simulador mente é conteúdo deste curso, não uma nota de rodapé.

SEM NOTA

R0 e R1 não valem nota. Existem para você errar antes de qualquer avaliação, e para absorver a folga das duas primeiras semanas de matrícula. Aproveite: gravar código errado num microcontrolador é gratuito e reversível, e essa é uma das poucas semanas do semestre em que também é indolor.

7 Previsão para o R0

PREVISÃO

Preencha antes da bancada. Vale o raciocínio, não o acerto.

P1. Com o kit ligado e a chave do painel de LEDs fechada, mas antes de gravar qualquer programa: os LEDs estarão acesos ou apagados? Justifique pelo estado dos pinos em reset.

P2. Seu programa usa dois laços contados encaixados como atraso. Estime o período do pisca em segundos e registre a base do palpite — antes de ajustar qualquer constante na bancada.

P3. Se você dobrar a constante do laço externo, o que acontece com o período? E com a frequência? Qual suposição sua resposta exige?

P4. O que você espera observar se trocar a ordem das duas linhas de inicialização — `TRISD` antes de `LATD`?

P5. Você grava e nada muda no painel. Liste, em ordem, as três primeiras coisas que verificaria — e diga o que cada verificação eliminaria.

BANCADA

Leve para a bancada: esta folha preenchida e o palpite de período do P2 anotado com o número, não "uns dois segundos". No encontro 2 você vai comparar esse número com uma conta, e no R3 com uma medida.

8 Exercícios

TAREFA

Exercício 0.1. Um colega afirma que o programa "acaba" quando `main` retorna.

(a) O que de fato acontece quando `main` retorna num programa de computador?

(b) E no microcontrolador, o que o processador faz depois da última instrução de `main`?

(c) Por que o `while (1)` não é questão de estilo?

Exercício 0.2. Sobre o atraso por laço contado, sem usar nenhum valor numérico de tempo: TAREFA

(a) Se a constante do laço externo dobrar, o que se pode afirmar sobre o período?

(b) Qual é a suposição por trás dessa afirmação?

(c) Por que não se pode afirmar o valor do período?

(d) Se o compilador removesse o laço, qual seria o sintoma observável no LED?

Exercício 0.3. Você compra um PIC18F4550 novo e monta na protoboard. TAREFA

(a) Existe equivalente ao SW9 nessa montagem?

(b) Que caminho de gravação serve, e o que ele exige fisicamente?

(c) Depois de gravar o chip da XM118 com um gravador, o caminho fácil continua disponível?

Exercício 0.4. Classifique cada item em **dentro do encapsulamento** ou **fora, na placa**: LED; conversor A/D; resistor de 330 Ω; LM35; temporizador; relé; EEPROM de 256 bytes; display; cristal de 20 MHz; conector USB. Justifique os dois casos que você achar ambíguos. TAREFA

Exercício 0.5. Um colega diz: “esse micro roda a 16 MHz, meu notebook roda a 3 GHz. É duzentas vezes pior.” Aponte três coisas que a comparação ignora, e diga em qual delas o micro é **melhor**, não apenas mais barato. TAREFA

No encontro 1: memória e folha de dados. A pergunta que abre a aula é uma conta que não fecha — o endereço de dados tem doze bits, e a instrução carrega oito. De onde saem os quatro que faltam? A resposta a essa pergunta é o mapa de memória inteiro, e é o que você vai implementar em Python no R2. NOTA